UNIVERSIDADE CATÓLICA DE SANTOS – UNISANTOS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

GUILHERME BORGES DA VEIGA
LEONARDO SILVA DOS SANTOS
MIGUEL REZENDE SILVA

DOCUMENTAÇÃO TÉCNICA DE ARQUITETURA DE SOFTWARE: SISTEMA DE TRIAGEM CLÍNICA E GESTÃO DE FILA DE PRIORIDADES EM PYTHON

SANTOS – SP
2026

GUILHERME BORGES DA VEIGA
LEONARDO SILVA DOS SANTOS
MIGUEL REZENDE SILVA

DOCUMENTAÇÃO TÉCNICA DE ARQUITETURA DE SOFTWARE: SISTEMA DE TRIAGEM CLÍNICA E GESTÃO DE FILA DE PRIORIDADES EM PYTHON

Relatório técnico e documentação de software apresentados à disciplina de Programação Orientada a Objetos, do Curso de Bacharelado em Ciência da Computação da Universidade Católica de Santos (UNISANTOS), como requisito parcial de avaliação prática e modelagem UML.

Repositório:
https://github.com/guizinbg/Trabalho_POO

SANTOS – SP
2026

SUMÁRIO

1 INTRODUÇÃO E ESCOPO DO SISTEMA

2 APLICAÇÃO DOS PILARES DE ORIENTAÇÃO A OBJETOS

2.1 HERANÇA E GENERALIZAÇÃO DE ENTIDADES

2.2 POLIMORFISMO NO FECHAMENTO FINANCEIRO

2.3 ENCAPSULAMENTO E VISIBILIDADE DE ATRIBUTOS

3 ARQUITETURA E ESPECIFICAÇÃO DO DIAGRAMA UML

3.1 RELAÇÕES DE COMPOSIÇÃO, AGREGAÇÃO E ASSOCIAÇÃO

3.2 DICIONÁRIO DE CLASSES, ATRIBUTOS E MÉTODOS

4 LÓGICA DE PRIORIZAÇÃO E FLUXO DE EXECUÇÃO

4.1 ESTRUTURA DA CHAVE DE PRIORIDADE E DESEMPATE ESTÁTICO

4.2 CICLO DE VIDA DO ATENDIMENTO CLÍNICO

5 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 INTRODUÇÃO E ESCOPO DO SISTEMA

Este documento descreve a arquitetura e a implementação do sistema desenvolvido para a disciplina de Programação Orientada a Objetos, cujo código-fonte encontra-se hospedado no repositório Trabalho_POO (https://github.com/guizinbg/Trabalho_POO). O software soluciona o problema de gerenciamento de atendimento médico e ordenação de filas de espera em clínicas e prontos-socorros, substituindo o modelo linear de atendimento por ordem estrita de chegada por uma fila dinâmica de prioridades.

O sistema controla quatro operações centrais do fluxo ambulatorial: o cadastro de pessoas (médicos e pacientes nas modalidades particular ou convênio), a aferição de sinais vitais na etapa de triagem, o posicionamento automático do paciente na fila conforme sua gravidade clínica e faixa etária, e o registro permanente do diagnóstico e da receita médica no prontuário individual.

Toda a modelagem foi escrita na linguagem Python seguindo o diagrama de classes da Unified Modeling Language (UML). A separação de responsabilidades garante que a regra de cálculo de risco clínico fique isolada na entidade de triagem, a estrutura de dados de ordenação permaneça encapsulada no gerenciador de fila e o faturamento varie de acordo com a subclasse do paciente atendido.

2 APLICAÇÃO DOS PILARES DE ORIENTAÇÃO A OBJETOS

A estrutura do projeto aplica diretamente os conceitos de Abstração, Herança, Polimorfismo e Encapsulamento. Em vez de concentrar a lógica em scripts procedurais ou dicionários soltos, cada elemento do domínio hospitalar foi abstraído em uma classe com estado interno e comportamentos próprios.

2.1 HERANÇA E GENERALIZAÇÃO DE ENTIDADES

A classe Pessoa atua como superclasse na base da árvore de herança. Ela armazena os atributos comuns a qualquer indivíduo cadastrado no sistema: nome (público, tipo string), idade (público, tipo int) e _cpf (privado, tipo string), além do método de leitura get_cpf(). A partir de Pessoa, derivam duas subclasses diretas: Medico e Paciente.

A subclasse Medico herda a estrutura de identificação de Pessoa e adiciona os atributos específicos da atividade médica: o registro profissional encapsulado _crm (string) e a especialidade espec (string), junto ao método operacional atender(). Na outra ramificação, a classe Paciente herda de Pessoa e introduz os atributos clínicos _sintomas (string) e _eh_retorno (bool).

Para tratar as diferentes regras de cobrança sem duplicar o código cadastral e clínico, Paciente funciona como superclasse para duas especializações: PacienteParticular e PacienteConvenio. Essa hierarquia em dois níveis permite que a fila de consultas e o prontuário manipulem objetos do tipo Paciente de maneira uniforme, independentemente de sua modalidade financeira.

2.2 POLIMORFISMO NO FECHAMENTO FINANCEIRO

O polimorfismo de sobrescrita (override) foi implementado no método gerar_conta(): float, declarado na classe Paciente e redefinido nas suas subclasses diretas:

- Na classe PacienteParticular: contém o atributo privado _valor_consulta (float). Quando o método gerar_conta() é chamado ao final do atendimento, ele calcula e retorna o valor financeiro cobrado diretamente do paciente, aplicando eventuais regras de retorno verificadas por is_retorno().
- Na classe PacienteConvenio: contém os atributos privados _nomePlano (string) e _numeroCarteirinha (string). A sobrescrita de gerar_conta() adapta o

retorno financeiro à cobertura do plano de saúde vinculado à carteirinha informada, diferenciando a fatura em relação ao atendimento particular.

2.3 ENCAPSULAMENTO E VISIBILIDADE DE ATRIBUTOS

No diagrama UML e no código em Python, o encapsulamento é demarcado pelo modificador de visibilidade privado/protegido (sinal de menos '-' no diagrama e prefixo de sublinhado '_' nos identificadores em Python). Essa restrição impede que trechos externos do programa alterem estados críticos sem passar pelas validações de negócio.

Os principais pontos de encapsulamento do projeto são: o documento _cpf em Pessoa, acessado exclusivamente via get_cpf(); a lista interna _fila na classe FilaConsultas, protegida para que inserções ocorram apenas pelo método enfileirar() seguido da rotina privada _ordenar(); o histórico _historico_consultas em Prontuario, manipulado somente por adicionar_consulta() e obter_historico(); e todas as aferições fisiológicas da classe Triagem (_temperatura, _pressaoSistolica, _nivel_dor, _cor_risco e _peso_prioridade).

3 ARQUITETURA E ESPECIFICAÇÃO DO DIAGRAMA UML

Além da hierarquia de herança, o diagrama de classes define como os objetos colaboram em tempo de execução por meio de associações diretas, agregações e composições.

3.1 RELAÇÕES DE COMPOSIÇÃO, AGREGAÇÃO E ASSOCIAÇÃO

O relacionamento entre Paciente e Prontuario é uma composição (representada pelo losango preenchido junto à classe Paciente). Isso significa que o ciclo de vida do objeto Prontuario está estritamente preso ao objeto Paciente: um prontuário não existe de forma isolada sem o respectivo paciente, sendo instanciado no momento em que o paciente é criado.

Já as classes FilaConsultas e Prontuario utilizam relações de agregação (losango vazio no diagrama). FilaConsultas agrega uma coleção de objetos ItemFila (_fila: List<ItemFila>), enquanto Prontuario agrega uma coleção de objetos Consulta (_historico_consultas: List<Consulta>). Na agregação, o objeto todo gerencia os objetos-parte, mas as partes podem transitar ou existir independentemente da estrutura agregadora (por exemplo, um ItemFila é retirado da fila ao ser desenfileirado, mas seus dados continuam existindo para gerar a consulta).

Por fim, as classes ItemFila e Consulta possuem associações diretas (setas simples de navegabilidade). Um ItemFila mantém referências para o Paciente e para a Triagem realizada. De modo análogo, o objeto Consulta referencia o Paciente atendido, o Medico responsável e a Triagem vinculada àquele atendimento.

3.2 DICIONÁRIO DE CLASSES, ATRIBUTOS E MÉTODOS

O Quadro 1 consolida todas as dez classes que compõem o diagrama UML do repositório Trabalho_POO, detalhando seus atributos com tipos e modificadores de acesso, bem como as assinaturas de seus métodos.

Quadro 1 – Especificação completa das classes, atributos e métodos do sistema

Classe	Relações UML	Atributos	Métodos e Responsabilidades
Pessoa	Superclasse base	+ nome: string + idade: int - _cpf: string	+ get_cpf(): string Retorna a string do CPF encapsulado.
Medico	Herda de Pessoa	- _crm: string + espec: string	+ atender(): string Registra a ação de

			atendimento clínico pelo profissional.
Paciente	Herda de Pessoa; Compõe Prontuario	- _sintomas: string - _eh_retorno: bool	+ is_retorno(): bool + get_prioridade_faixa_etaria(): tuple + gerar_conta(): float Verifica retorno, calcula prioridade demográfica por idade e define interface de cobrança.
PacienteParticular	Herda de Paciente	- _valor_consulta: float	+ gerar_conta(): float Sobrescreve o cálculo financeiro para cobrança particular.
PacienteConvenio	Herda de Paciente	- _nomePlano: string - _numeroCarteirinha: string	+ gerar_conta(): float Sobrescreve o cálculo financeiro considerando o plano e carteirinha.
Triagem	Associada por ItemFila e Consulta	- _temperatura: float - _pressaoSistolica: int - _nivel_dor: int - _cor_risco: string - _peso_prioridade: int	+ calcular_risco() + exibir_triagem(): string Avalia sinais vitais, atribui a cor do protocolo e define o peso inteiro de prioridade.
ItemFila	Agregada por FilaConsultas; Associa Paciente e Triagem	- _contador_chegada: int (static) + ordem_chegada: int + data_hora_chegada: datetime + desc_idade: string + chave_prioridade: tuple	+ calcular_tempo_estimado(posicao_fila: int): tuple Encapsula o paciente na fila, controla o contador estático de chegada e monta a tupla de ordenação.
FilaConsultas	Agrega ItemFila	- _fila: List<ItemFila>	+ enfileirar(paciente: Paciente, triagem: Triagem): ItemFila - _ordenar() + desenfileirar(): ItemFila + proximo(): ItemFila + esta_vazia(): bool + tamanho(): int + obter_lista(): list Gerencia a inserção, ordenação automática e remoção de pacientes da espera.
Prontuario	Composta por Paciente; Agrega Consulta	- _historico_consultas: List<Consulta>	+ adicionar_consulta(consulta: Consulta) + obter_historico():

			List<Consulta> + exibir_prontuario_texto(): string Mantém e formata o histórico clínico completo do paciente.
Consulta	Agregada por Prontuario; Associa Medico, Paciente e Triagem	- _data_hora: datetime - _diagnostico: string - _receita: string	+ obter_resumo_texto(): string Consolida os dados do atendimento realizado, diagnóstico médico e prescrição.

Fonte: Elaborado pelos autores com base no diagrama de classes do repositório Trabalho_POO (2026).

4 LÓGICA DE PRIORIZAÇÃO E FLUXO DE EXECUÇÃO

O diferencial técnico do sistema está na maneira como a fila de espera resolve conflitos de prioridade sem perder a ordem cronológica de chegada para casos de mesma urgência.

4.1 ESTRUTURA DA CHAVE DE PRIORIDADE E DESEMPATE ESTÁTICO

Quando um paciente passa pela triagem, o método calcular_risco() avalia os atributos _temperatura, _pressaoSistolica e _nivel_dor para definir a _cor_risco (seguindo o padrão hospitalar de classificação por cores, como Vermelho, Laranja, Amarelo, Verde e Azul) e atribuir um inteiro a _peso_prioridade. Paralelamente, o método get_prioridade_faixa_etaria() na classe Paciente verifica o atributo herdado idade para conceder prioridade legal a idosos ou crianças de colo, retornando uma tupla com o peso demográfico e a descrição da faixa.

Ao empacotar esses objetos dentro de um ItemFila, a classe utiliza o atributo estático _contador_chegada: int (static). Por ser uma variável de classe (estática), seu valor é compartilhado por todas as instâncias de ItemFila e incrementado a cada novo paciente admitido, gravando um número sequencial único no atributo de instância ordem_chegada.

Com esses três dados, ItemFila monta o atributo chave_prioridade: tuple, que combina: (1) o peso de risco clínico da Triagem, (2) o peso da faixa etária do Paciente e (3) o inteiro sequencial ordem_chegada. No Python, a comparação entre tuplas ocorre elemento por elemento da esquerda para a direita. Assim, quando o método privado _ordenar() da classe FilaConsultas executa a ordenação da lista _fila, o sistema posiciona automaticamente os casos de maior gravidade clínica no início; se dois pacientes tiverem a mesma cor de triagem, o critério de desempate passa a ser a faixa etária; se o empate persistir, prevalece quem chegou primeiro (menor ordem_chegada). Além disso, o método calcular_tempo_estimado(posicao_fila: int) calcula a previsão de espera com base na posição atual do item na lista ordenada.

4.2 CICLO DE VIDA DO ATENDIMENTO CLÍNICO

O fluxo completo de execução do programa segue cinco etapas sequenciais:

1. Instanciação Cadastral: O sistema cria o objeto Medico com seu _crm e espec, e instancia o paciente como PacienteParticular ou PacienteConvenio. No construtor de Paciente, o objeto Prontuario é inicializado vazio.
2. Aferição na Triagem: Um objeto Triagem é instanciado com os sinais vitais colhidos (_temperatura, _pressaoSistolica e _nivel_dor). O método calcular_risco() é invocado para preencher _cor_risco e _peso_prioridade.
3. Enfileiramento e Ordenação: O método enfileirar(paciente, triagem) de FilaConsultas instancia o ItemFila, incrementa _contador_chegada, registra data_hora_chegada, gera a chave_prioridade e aciona _ordenar() para atualizar a posição de todos os aguardantes.
4. Chamada de Consultório e Consulta: Através de proximo() ou desenfileirar(), o primeiro ItemFila é extraído da fila. O médico executa atender() e um objeto Consulta é instanciado registrando _data_hora, _diagnostico e _receita, mantendo os ponteiros para Medico, Paciente e Triagem.
5. Arquivamento no Prontuário e Conta: O objeto Consulta recém-criado é inserido no Prontuario do paciente via adicionar_consulta(). O resumo pode ser impresso com obter_resumo_texto() ou exibir_prontuario_texto(), e a cobrança final é emitida pela chamada polimórfica gerar_conta().

5 CONSIDERAÇÕES FINAIS

A implementação contida no repositório Trabalho_POO cumpre os requisitos da modelagem orientada a objetos ao separar os dados cadastrais, a triagem clínica, a estrutura de fila e o registro médico em dez classes coesas. O uso de tuplas comparáveis (chave_prioridade) aliadas a um contador estático (_contador_chegada) na classe ItemFila resolve a fila de prioridades de forma determinística, enquanto a árvore de herança iniciada em Pessoa garante reuso de código e flexibilidade no cálculo financeiro das consultas.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: Informação e documentação — Referências — Elaboração. Rio de Janeiro: ABNT, 2018.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2006.

LUTZ, Mark. Aprendendo Python. 5. ed. Porto Alegre: Bookman, 2015.

VEIGA, Guilherme Borges da; SANTOS, Leonardo Silva dos; SILVA, Miguel Rezende. Trabalho_POO: Sistema de Triagem e Gestão de Consultas em Python. Santos: GitHub, 2026. Disponível em: https://github.com/guizinbg/Trabalho_POO. Acesso em: 13 nov. 2026.